

Protocolos de comunicación

De los cables a la red: de bajo a alto nivel

Luis Diego García Rojas

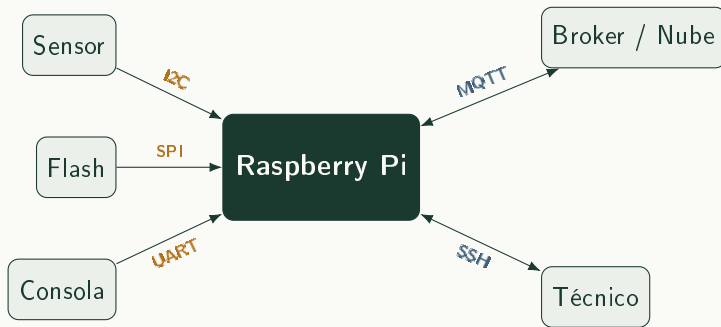
II Semestre 2026

EL5841 — Taller de Sistemas Embebidos

Instituto Tecnológico de Costa Rica

El sistema que nos va a acompañar

Un sistema de control de acceso sobre Raspberry Pi 4 con Linux embebido.



Cada protocolo de hoy resuelve una de estas conexiones. Al final volvemos a este diagrama.

Bajo nivel

entre chips, capas 1–2

- UART / USART
- SPI
- I2C
- JTAG (interfaz de prueba y depuración)

Alto nivel

entre sistemas, capa 7

- SSH
- MQTT

Hilo conductor

Subimos por las capas del modelo OSI y seguimos la vida del dispositivo: desde el primer arranque hasta la operación en campo.

El modelo OSI

¿Qué es un protocolo? ¿Por qué en capas?

Un **protocolo** es un conjunto de reglas acordadas para comunicarse.

Se organizan en **capas**: cada una resuelve un problema y confía en la de abajo, sin necesitar saber cómo funciona.

Cambiar el WiFi por un cable no obliga a reescribir la aplicación: solo cambia la capa de abajo.

Idea clave

Los protocolos de **bajo nivel** mueven *bits entre chips*.

Los de **alto nivel** mueven *mensajes entre sistemas*, apoyados en TCP/IP.

Dónde vive cada protocolo

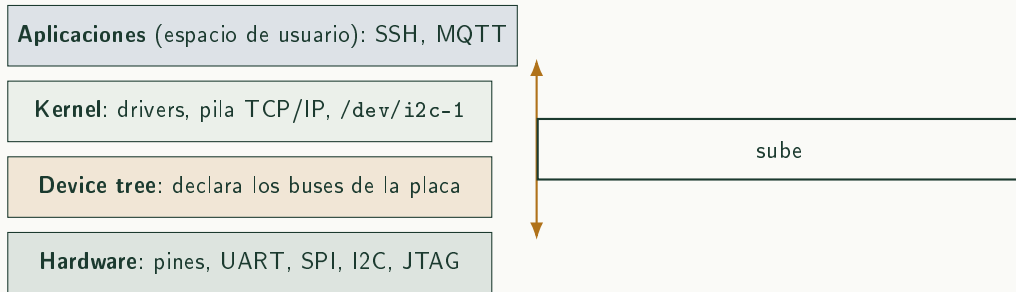
Capa OSI	Protocolos de hoy
7 — Aplicación	SSH, MQTT
4 — Transporte	TCP (<i>base de ambos</i>)
3 — Red	IP
2 — Enlace	UART, SPI, I2C · JTAG*
1 — Física	

* Aclaración honesta

JTAG **no** es un protocolo de datos como los demás: es una interfaz de *prueba y depuración*. Lo incluimos porque es esencial en el ciclo de vida del sistema.

Este mapa es el índice: vamos a volver a él al iniciar cada sección.

OSI y el stack de Linux embebido



Los protocolos de bajo nivel se tocan desde *drivers* o nodos en /dev; SSH y MQTT, desde *aplicaciones*.

**Bajo nivel: comunicación entre
chips**

Dos ejes que ordenan todo el bloque

¿Hay reloj compartido?

Asíncrono (UART): no.

Síncrono (SPI, I2C): sí.

¿Punto a punto o bus?

Punto a punto (UART, SPI).

Bus (I2C): muchos en dos líneas.

UART → **SPI** → **I2C**

de lo más simple a lo más complejo

Fundamentos

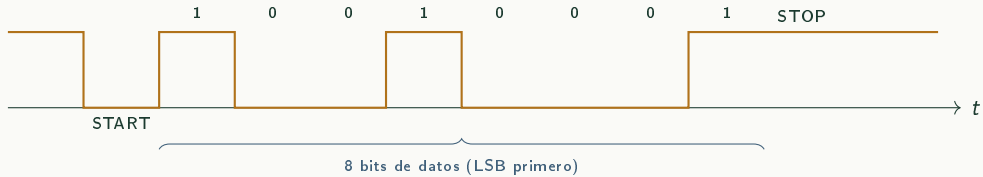
- Asíncrono, sin reloj. Dos líneas: TX y RX.
- Trama: bit de inicio, datos, paridad opcional, bit de parada.
- Ambos lados acuerdan el **baud rate**.
- USART agrega un modo síncrono.

Punto complejo: sin reloj, el receptor detecta el flanco del bit de inicio y muestrea a mitad de cada bit (sobremuestreo $\times 16$). De ahí la tolerancia de $\pm 2-3\%$ en el baud rate.

Ejemplos y casos

- Consola de depuración, GPS, módulos Bluetooth.
- **Industrial:** UART sobre RS-485 es la base de *Modbus RTU* en PLCs y variadores.

UART: la trama en el tiempo



Se transmite primero el bit menos significativo. La línea en reposo está en alto.

Fundamentos

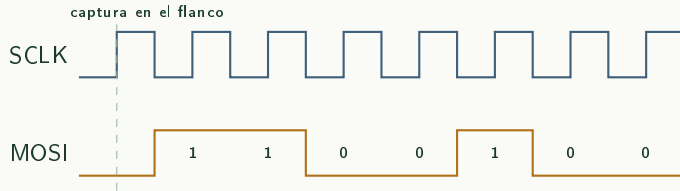
- Síncrono y full-duplex.
- Líneas: SCLK, MOSI, MISO y un CS por periférico.
- Un controlador, varios periféricos. Muy rápido (decenas de MHz).

Ejemplos y casos

- Memorias flash, tarjetas SD, pantallas, ADC de alta velocidad.

Punto complejo: los 4 modos (CPOL/CPHA) definen en qué nivel reposa el reloj y en qué flanco se captura el dato. Es la causa #1 de “conecté todo y no funciona”.

SPI: el reloj sincroniza cada bit



El receptor no adivina el ritmo: el reloj le dice exactamente cuándo leer.

Fundamentos

- Dos líneas (SDA, SCL) compartidas por muchos dispositivos.
- Direccionamiento de 7 bits, ACK/NACK.
- 100 kHz, 400 kHz, 1 MHz.

Punto complejo: lógica **open-drain con resistencias pull-up**: nadie “empuja” un 1, solo suelta la línea. Habilita el arbitraje multi-master y el *clock stretching*, pero la capacitancia del bus limita la velocidad.

Ejemplos y casos

- Sensores de temperatura y aceleración, EEPROM, RTC, gestión de energía.

Comparativa del bloque de bajo nivel

	UART	SPI	I2C
Reloj	Asíncrono	Síncrono	Síncrono
Líneas	2	3 + 1/disp.	2
Dispositivos	2	Varios (CS)	Muchos (dirección)
Velocidad	Baja-media	Muy alta	Media
Dúplex	Full	Full	Half
Uso típico	Consola, GPS	Flash, pantalla	Sensores

Esta es, casi siempre, la diapositiva que más gente fotografía.

JTAG — ¿y cómo programamos y probamos el chip?

Fundamentos

- Estándar IEEE 1149.1. Líneas TCK, TMS, TDI, TDO.
- Nació para probar placas (*boundary scan*); hoy también programa y depura.
- SWD: alternativa de 2 pines en ARM.

Ejemplos y casos

- Prueba de soldaduras en producción sin sondas físicas.
- Programación de FPGA, depuración paso a paso.

Punto complejo: la **máquina de estados del TAP** (16 estados manejados solo con TMS) y el encadenamiento de varios chips (*daisy chain*).

Puente a seguridad

Un JTAG habilitado en un producto comercial permite *extraer el firmware*: es un vector de ataque real. Nos lleva de la mano a SSH.

Transición: de los cables a la red

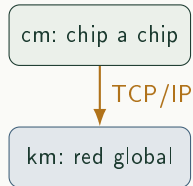
¿Por qué no alcanza con lo anterior?

UART, SPI e I2C funcionan en **centímetros**, entre chips de una placa.

Para llegar a un servidor al otro lado del mundo hace falta:

- Direccionamiento global — **IP**
- Entrega confiable — **TCP**
- Seguridad — **cifrado y autenticación**

Por eso SSH y MQTT viven *sobre* TCP/IP, en la capa de aplicación.



**Alto nivel: comunicación entre
sistemas**

Fundamentos

- Cliente-servidor, puerto 22.
- Canal cifrado y autenticado sobre una red insegura.
- Reemplazó a Telnet (contraseñas en texto plano).

Punto complejo: acordar una clave secreta sin haberse visto (**Diffie-Hellman**); verificar la identidad del servidor con su *host key* (la “huella” de la primera conexión); autenticar por llave pública en vez de contraseña.

Ejemplos y casos

- Administración remota de gateways y Raspberry Pi.
- Actualización con SCP/SFTP, Git, túneles.

Fundamentos

- Modelo **publicar/suscribir** con un **broker** en el centro.
- Los dispositivos hablan por *tópicos*:
planta1/motor3/vibracion.
- Liviano, para redes inestables. Puerto 1883, o 8883 con TLS.

Punto complejo: niveles de **QoS** (0: a lo sumo una vez; 1: al menos una vez; 2: exactamente una vez), *retained messages* y *Last Will* (aviso de desconexión inesperada).

Ejemplos y casos

- Monitoreo industrial (IIoT), domótica.
- AWS IoT, Azure IoT, Mosquitto. Sparkplug B para SCADA.

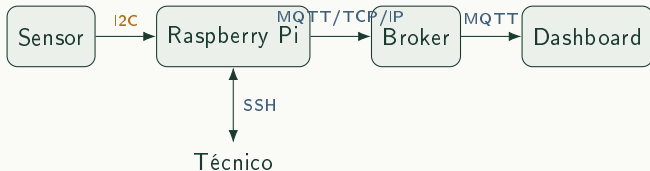
SSH frente a MQTT

	SSH	MQTT
Modelo	Uno a uno	Muchos a muchos
Propósito	<i>Controlar</i> un equipo	<i>Mover</i> datos
Patrón	Sesión interactiva	Publicar / suscribir
Intermediario	No	Broker

No compiten: resuelven problemas distintos de la misma capa.

Cierre y actividad

El recorrido completo de un evento



Mensaje central

No hay un protocolo “mejor”. Cada uno resuelve el problema de su capa, y un sistema real los necesita todos.

La misión: laboratorio cerrado

Un corte de luz bloqueó el laboratorio. La contraseña del servidor quedó **dividida en dos**.

1. **I2C (Tinkercad)**: el sensor de temperatura guarda el **fragmento 1**. Se libera al llegar a 30 °C.
2. **UART (Tinkercad)**: programan paso a paso el lector y la cerradura. Al abrir, revela el **fragmento 2**.
3. **SSH (en casa, con Docker)**: construyen el servidor archivo por archivo, entran con los dos fragmentos, abren la puerta y descargan el acta.

Cada protocolo abre una parte del camino: si uno falla, la puerta sigue cerrada.

Referencias

- H. Zimmermann, “OSI Reference Model—The ISO Model of Architecture for Open Systems Interconnection”, *IEEE Transactions on Communications*, vol. 28, no. 4, pp. 425–432, 1980.
- NXP Semiconductors, *UM10204: I2C-bus specification and user manual*, Rev. 7.0, 2021.
- IEEE, *IEEE Std 1149.1: Standard for Test Access Port and Boundary-Scan Architecture*.
- T. Ylonen y C. Lonvick, *RFC 4251: The Secure Shell (SSH) Protocol Architecture*, IETF, 2006.
- T. Ylonen y C. Lonvick, *RFC 4253: The Secure Shell (SSH) Transport Layer Protocol*, IETF, 2006.
- OASIS, *MQTT Version 5.0*, OASIS Standard, 2019.
- Microchip Technology, *ATmega328P Datasheet* (USART, SPI y TWI).
- Analog Devices, *TMP35/TMP36/TMP37 Low Voltage Temperature Sensors*, hoja de datos.
- Arduino, documentación de referencia de las librerías `Wire` y `SoftwareSerial`, [arduino.cc](https://www.arduino.cc).
- OpenBSD Project, páginas de manual de OpenSSH: `ssh(1)`, `sshd_config(5)`, `scp(1)`.

¿Preguntas?

Del cable al servidor, capa por capa.